

# Projeto 3 - Quicksort e Quicksort2

## ATP II

### 1 Definição

O algoritmo do quicksort pode ser implementado eficientemente de forma recursiva. Apesar disso, estudos apresentaram variações na forma de definição do elemento pivô utilizado a cada chamada da função, buscando acelerar o processo de ordenação. Uma dessas variações, que chamaremos aqui de Quicksort2, usa dois pivôs em vez de apenas um.

Para o uso de dois pivôs, o que se faz é separar os elementos do vetor em três conjuntos, um com elementos menores que o menor pivô, outro com elementos maiores que o maior pivô e um terceiro conjunto com elementos maiores ou iguais ao menor pivô e menores ou iguais ao maior pivô. Após a separação são feitas as chamadas recursivas para cada um desses conjuntos.

### 2 O que deve ser feito

Implemente dois programas para fazer a ordenação de vetores de palavras. O primeiro deles deve ser o Quicksort convencional, com um pivô, e o segundo deve ser o Quicksort2 descrito acima. Esses dois programas devem ser executados e ter seu tempo de execução medido diversas vezes, usando o procedimento a seguir:

1. Os vetores de entrada estão disponíveis no classroom;
2. Para cada vetor de entrada execute cada um dos programas por cinco vezes, obtendo o tempo médio de execução para cada arquivo de entrada;
3. O arquivo ordenado deve ser salvo em um arquivo com nome igual ao de entrada, trocando “.input” por “.output”;
4. Os tempos devem ser medidos usando a função `clock_gettime()`, como visto no exemplo a seguir:

```
// declarando variáveis utilizadas (iterations= 1 milhão)
    struct timespec begin, end;
    long seconds, nanoseconds;
    double elapsed;
// anota o tempo de início em begin
    clock_gettime(CLOCK_REALTIME, &begin);

    for (int i=0; i<iterations; i++) { qualquerfuncao(i); }
```

```
// anota o tempo de término em end
    clock_gettime(CLOCK_REALTIME, &end);
    seconds = end.tv_sec - begin.tv_sec;
    nanoseconds = end.tv_nsec - begin.tv_nsec;
    elapsed = seconds + nanoseconds*1e-9;
    printf("Time measured: %.3f seconds.\n", elapsed);
```

### 3 Entrada de dados

O vetor a ser ordenado está contido em artigos que estarão disponíveis no classroom.

### 4 Saída de dados

Seu programa deve produzir um arquivo com o vetor ordenado e medir o tempo gasto para realizar a ordenação usando os dois métodos do quicksort.

### 5 Entrega

Entregar o código fonte do programa, devidamente comentado, no classroom e um relatório analisando os tempos gastos na ordenação.

**PRAZO:** 08/12, até 23h59 no classroom.